

ADDIS ABABA UNIVERSITY

School of Information Technology & Engineering · Department of Software Engineering
Mobile Application Development · 4th Year, Semester 2

ASSIGNMENT 2**Networking, REST APIs & Data Handling in Flutter**

Unit 4 · 10% of Final Course Grade

Course	Mobile Application Development (Undergraduate)
Unit	Unit 4 — Networking, REST APIs and Data Handling in Flutter
Assignment	Individual Programming Assignment
Weight	10% of Final Course Grade (100 marks total, scaled to 10%)
Issue Date	<u>April -22-2026</u>
Due Date	_____ (12 days from issue date)
Submission	GitHub repository link + APK/screen recording via Google Classroom
Instructor	Abel Tadesse
Late Penalty	2.5 marks deducted per day late. No submissions accepted after 5 days.

1. Overview and Purpose

In this assignment you will build a fully functional Flutter application that communicates with a real public REST API over the internet. You will demonstrate mastery of the core Unit 4 concepts: making HTTP requests, handling asynchronous code, parsing JSON into typed Dart model classes, building a clean API service layer, and managing the three UI states (loading, error, and data) correctly.

You will choose ONE application from the three tracks listed in Section 3. All three tracks share the same technical requirements but use different public APIs, allowing you to work on a topic that interests you.

2. Learning Outcomes

Upon successful completion of this assignment you will be able to:

- Configure the http package and make GET and POST HTTP requests in Flutter.
- Parse JSON responses into typed Dart model classes using factory fromJson constructors.
- Manage asynchronous operations correctly using async/await and Future<T>.
- Build a clean, reusable API service layer separated from the UI layer.
- Handle all three UI states (loading spinner, error message, data display) in a FutureBuilder.

- Implement robust error handling for `SocketException`, `TimeoutException`, and non-200 HTTP responses.
- Securely manage API keys using environment variables or a `.env` file excluded from version control.

3. Application Tracks — Choose ONE

Select one track below. All tracks are equally weighted and graded on the same rubric. Choose based on your interest.

TRACK A — Country Explorer App (API: RestCountries)

API Base URL: <https://restcountries.com/v3.1> (free, no API key required)

Description: Build an app that lets users browse and search countries of the world.

Required Screens:

- Home screen: displays a scrollable list of ALL countries (name, flag emoji, region)
- Search screen: lets the user type a country name and fetches matching results via `/name/{name}`
- Detail screen: shows full details of a selected country (capital, population, currencies, languages, area, timezones)

Required API Endpoints:

- GET `/all?fields=name,flags,region,population` — fetch all countries
- GET `/name/{name}` — search by name
- GET `/alpha/{code}` — fetch a single country by ISO code (for the detail screen)

Model class required: Country (name, flagEmoji, region, capital, population, currencies, languages, area, timezones, alpha3Code)

TRACK B — News Reader App (API: NewsAPI)

API Base URL: <https://newsapi.org> (free developer tier — register at newsapi.org for a free key)

Description: Build a news reader that fetches top headlines and lets users search by keyword.

Required Screens:

- Home screen: displays top headlines from a user-selected country (dropdown of at least 5 countries)
- Search screen: lets the user search for articles by keyword across all sources
- Article detail screen: shows the full article title, description, source, author, date, and a 'Read Full Article' button (url_launcher to open the URL)

Required API Endpoints:

- GET `/v2/top-headlines?country={cc}&apiKey={key}` — top headlines
- GET `/v2/everything?q={query}&apiKey={key}&pageSize=20` — keyword search

Model class required: Article (title, description, url, urlToImage, sourceName, author, publishedAt)

⚠ Security: Your API key must NOT be committed to GitHub. Store it in a `.env` file and load it using the `flutter_dotenv` package. Add `.env` to `.gitignore`.

TRACK C — Recipe Browser App (API: TheMealDB)**API Base URL:** <https://www.themealdb.com/api/json/v1/1> (completely free, no key required)**Description:** Build a recipe browser that allows users to search and explore meal recipes.**Required Screens:**

- Home screen: displays a list of meal categories (each with thumbnail image and description)
- Category screen: shows all meals within a selected category (thumbnail + meal name)
- Meal detail screen: full recipe including ingredients list with measures, instructions, and YouTube link (url_launcher)

Required API Endpoints:

- GET /categories.php — fetch all categories
- GET /filter.php?c={category} — meals in a category
- GET /lookup.php?i={mealId} — full meal details

Model classes required: MealCategory (idCategory, strCategory, strCategoryThumb, strCategoryDescription) and Meal (idMeal, strMeal, strMealThumb, strCategory, strArea, strInstructions, strYoutube, ingredients)**4. Technical Requirements (Mandatory for ALL Tracks)**

The following requirements are non-negotiable and will be checked during grading. Failure to meet any mandatory requirement may result in zero marks for the related criterion.

4.1 HTTP & Networking

- **Use the http package:** All HTTP calls must use the http package (not Dio or any other HTTP library). Import it as: `import 'package:http/http.dart' as http;`
- **URI construction:** Always construct URIs using `Uri.https()` or `Uri.parse()` — never concatenate strings directly.
- **Status code check:** Check `response.statusCode == 200` before parsing. Throw a custom `ApiException` for non-200 responses.
- **Timeout:** Set a timeout of 10 seconds on all requests using `.timeout(const Duration(seconds: 10))`.
- **Headers:** Set the Content-Type and Accept headers to 'application/json' where applicable.

4.2 Dart Model Classes

- **factory fromJson:** Every model class must have a factory `Name.fromJson(Map<String, dynamic> json)` constructor. No dynamic types in the UI layer.
- **toJson:** Every model class must have a `Map<String, dynamic> toJson()` method.
- **Explicit casts:** Use explicit type casts (as String, as int) — never rely on dynamic. Handle nullable fields with null-aware operators.
- **Immutability:** All model fields must be declared final. Provide a `copyWith()` method for any model that may be updated.
- **Separate file:** Each model class must be in its own file under `lib/models/`.

4.3 API Service Layer

You must implement a dedicated API service class (e.g., CountryApiService, NewsApiService, MealApiService) that:

- Is placed in lib/services/ and contains ALL HTTP logic (no HTTP calls outside this class)
- Declares a private final String _baseUrl field
- Sets a _timeout field (Duration — 10 seconds)
- Declares shared _headers as a Map<String, String>
- Has a private _checkResponse(http.Response) method that throws ApiException for non-200 status codes
- Has one method per API endpoint (e.g., fetchAllCountries(), searchByName(String name), fetchByCode(String code))
- Returns typed objects (Future<List<Country>>, Future<Country>) — never Future<dynamic>

4.4 Async Programming

- **async/await:** All asynchronous operations must use async/await syntax (not .then()/.catchError()) chains in the service layer).
- **No async in build():** Never call an async function directly inside the build() method. Use FutureBuilder or call the function in initState().
- **mounted check:** After any await operation inside a StatefulWidget, check if (mounted) before calling setState().
- **FutureBuilder:** At least one screen must use FutureBuilder<T> with all four states handled: ConnectionState.waiting, error, no data, and data.

4.5 Error Handling

Your app must gracefully handle all of the following error conditions:

Error Type	Required Behaviour
<code>SocketException</code>	No internet connection — show a user-friendly 'No internet connection' message with a Retry button
<code>TimeoutException</code>	Request took too long — show 'Request timed out. Please try again.'
<code>ApiException</code> (custom)	Server returned a non-200 status — show the HTTP status code and a descriptive message
<code>FormatException</code>	Malformed JSON — show 'Unexpected data format received'
<code>Generic Exception</code>	Catch-all — show 'An unexpected error occurred: {message}'

4.6 Project Structure

Your project must follow this folder structure exactly:

```
lib/
├── main.dart
├── models/
│   └── <ModelName>.dart          # One file per model
├── services/
│   ├── <Name>_api_service.dart    # All HTTP logic
│   └── api_exception.dart        # Custom exception class
├── providers/ (optional)
│   └── <Name>_provider.dart       # ChangeNotifier if used
├── screens/
│   └── home_screen.dart
```

```
└─ <second>_screen.dart
└─ detail_screen.dart
```

5. Deliverables

Submit ALL of the following by the due date. Incomplete submissions will be penalised.

D1. GitHub Repository — A public GitHub repository containing the complete Flutter project source code. The repository must have a meaningful name (e.g., flutter-country-explorer), include a README.md (see below), and have at least 5 meaningful commits with descriptive commit messages. The repository must NOT contain any API keys.

D2. README.md — A README.md file in the root of your repository containing: (1) your name and student ID, (2) the track you chose, (3) a brief description of the app, (4) instructions for running the app locally (including how to set up the .env file if applicable), (5) a list of all API endpoints used, (6) a list of known limitations or bugs.

D3. Screen Recording — A screen recording (MP4, minimum 2 minutes) demonstrating: the app loading data from the API, the search functionality working, navigating to a detail screen, the error state (turn on Airplane Mode while the app is running and show the error message), and the Retry button bringing data back after re-enabling internet.

D4. Submission Form — Submit the GitHub URL and screen recording link via Google Classroom. The submission form must include your full name, student ID, and the track you chose.

6. Grading Rubric (100 marks total → 10% of course)

Each criterion is marked out of the marks shown. The total is 100 marks which is then scaled to 10% of your final course grade.

Criterion	Excellent (Full)	Satisfactory (Partial)	Needs Work (0)	Marks
HTTP Requests & API Setup <i>Marks 1–20</i>	http package used correctly; Uri.https() for all URLs; correct headers; 10s timeout set; all required endpoints called; api_exception.dart custom class present	Most endpoints work but 1–2 missing; timeout or headers partially implemented; minor URL construction issues	HTTP not working or using wrong library; hardcoded URL strings; no timeout	20
Model Classes & JSON Parsing <i>Marks 1–20</i>	All required model classes present with fromJson factory, toJson, and copyWith; all fields are final; explicit type casts; nullable fields handled correctly with ?. and ??	fromJson present but missing toJson or copyWith; some dynamic types used; minor nullable field issues	No model classes; JSON parsed inline in the UI; using dynamic throughout	20
API Service Layer Architecture <i>Marks 1–20</i>	Dedicated service class in lib/services/; all HTTP logic inside it; _baseUrl, _timeout, _headers, _checkResponse	Service class exists but some HTTP calls leak into UI; _checkResponse missing; minor structural issues	No service class; all HTTP code in widgets/screens; no separation of concerns	20

	implemented; UI has zero HTTP imports; typed return types			
Error Handling & FutureBuilder <i>Marks 1–20</i>	All 5 error types caught and shown with user-friendly messages; Retry button works; FutureBuilder handles all 4 states (waiting/error/null/data); mounted check present after async gaps	2–3 error types handled; FutureBuilder present but 1–2 states missing; no Retry button or mounted check	No try/catch; app crashes on network errors; FutureBuilder missing or has only data state	20
UI/UX & App Functionality <i>Marks 1–10</i>	All 3 required screens present and functional; list shows thumbnail/image and key fields; detail screen is complete; navigation works; smooth loading indicators; responsive layout	2 of 3 screens complete; images loading but some fields missing; navigation works	Only 1 screen; mostly static/hardcoded data; navigation broken	10
Code Quality & Project Structure <i>Marks 1–5</i>	Correct folder structure (lib/models/, lib/services/, lib/screens/); meaningful variable names; no dead code; const used correctly; files named in snake_case	Minor structure deviations; some naming issues; a few unused imports	Everything in main.dart; no structure; poor naming	5
Version Control & Submission <i>Marks 1–5</i>	Public GitHub repo; ≥5 meaningful commits; README.md complete (all 6 items); no API keys in repo; screen recording shows all required scenarios	GitHub repo present; 2–4 commits; README partially complete; recording present but missing error demo	No GitHub repo; or single commit; no README; no recording	5

TOTAL (scaled to 10% of course grade)	100
--	------------

7. Bonus Tasks (+15 marks, max 100)

These tasks are optional. They cannot bring your total above 100 marks, but they provide an opportunity to demonstrate advanced skills and recover marks lost elsewhere.

+5 marks Pagination — Implement load-more / infinite scroll pagination in your list screen. Each page loads 10–20 items. A 'Load More' button or automatic trigger on scroll must work correctly with the API.

+5 marks Local Caching — Cache the last successful API response in memory with a TTL (time-to-live) of 5 minutes. On re-launch within TTL, show cached data instantly while refreshing in the background. Implement a visible 'Cached' badge when data is served from cache.

+5 marks Search Debouncing — In the search screen, debounce the search input so the API is not called on every keystroke. Use a Timer to delay the API call by 400ms after the user stops typing. Show a circular indicator while debounced calls are in progress.

8. Academic Integrity & Collaboration Policy

IMPORTANT — Read Carefully

This is an individual assignment. The following rules apply:

- You must write all code yourself. Submitting another student's code as your own is plagiarism and will result in zero marks for both parties.
- You may use official documentation, Stack Overflow, and the course lecture code examples as references, but you must understand and be able to explain every line of code you submit.
- AI-assisted coding tools (GitHub Copilot, ChatGPT, etc.) are permitted as a learning aid, but you are responsible for understanding the code. If asked during viva/oral exam, you must explain any part of your submission.
- If you use any external code or tutorial as a reference, cite it in a comment in your code and in your README.md. Failure to cite is treated as plagiarism.
- Do NOT share your code with other students — even 'just to help'. Both parties will receive zero marks if identical code is found.

Suspected plagiarism cases will be referred to the University Academic Integrity Committee.

9. Tips for Success

- **Start with the API service:** Before building any UI, test all your API calls in Dart (main.dart or a test file) to confirm the responses parse correctly. A broken service class will cause everything downstream to fail.
- **Use JSONPlaceholder first:** If you are struggling with API integration, start with <https://jsonplaceholder.typicode.com/posts> to practise — it requires no key and returns simple JSON.
- **Error handling saves marks:** Error handling is worth 20 marks. Prioritise it over UI polish. Turn on Airplane Mode frequently while testing to verify your error states work correctly.
- **Commit early and often:** Make your first GitHub commit with just main.dart, then commit after each feature (model class, service method, screen). This shows your work process and protects against accidental data loss.
- **Never store keys in code:** If your track requires an API key, store it in assets/.env and use flutter_dotenv. Add .env to .gitignore immediately — GitHub scans for exposed API keys and may invalidate them.
- **const keyword matters:** Use const wherever possible on widgets and model instantiation. It is worth marks under code quality and demonstrates your understanding of Flutter's optimisation model.

10. Submission Checklist

Use this checklist before submitting. Each item that is missing may cost you marks.

Code & Architecture

- ☐ http package used for all requests (not Dio)
- ☐ All URIs built with `Uri.https()` or `Uri.parse()` — no string concatenation
- ☐ 10-second timeout on every request
- ☐ All required model classes present with `fromJson`, `toJson`, `copyWith`
- ☐ All model fields are final and explicitly typed (no dynamic)
- ☐ Dedicated API service class in `lib/services/`
- ☐ No HTTP imports in any Screen or Widget file
- ☐ `_checkResponse()` method throws `ApiException` for non-200 responses
- ☐ `FutureBuilder` handles all 4 states (waiting, error, no-data, data)
- ☐ Retry button works when network connection is restored

Error Handling

- ☐ `SocketException` caught and shows 'No internet connection'
- ☐ `TimeoutException` caught and shows 'Request timed out'
- ☐ Non-200 status codes caught by `ApiException`
- ☐ `FormatException` caught for malformed JSON
- ☐ Generic catch-all Exception handler present

Project Structure

- ☐ `lib/models/` folder with one file per model
- ☐ `lib/services/` folder with service class and `api_exception.dart`
- ☐ `lib/screens/` folder with all screen files
- ☐ All files named in snake_case

Submission

- ☐ GitHub repository is public
- ☐ At least 5 commits with meaningful messages
- ☐ `README.md` includes all 6 required items (name, ID, track, description, setup, limitations)
- ☐ No API keys committed to the repository
- ☐ `.env` file in `.gitignore` (if applicable)
- ☐ Screen recording shows: loading, search, detail, error state, retry
- ☐ Google Classroom submission form submitted with correct GitHub URL

Prepared by:

Abel Tadesse

School of IT & Engineering, Addis Ababa University